CivicPulse: speaker script

Four speakers, ten slides, about eight minutes, then questions

Speaker	Slides	Covers	Time
Speaker 1	Slides 1–3	Title, problem, solution	≈ 2 min 10 s
Speaker 2	Slides 4–5	Architecture, smart grouping	≈ 2 min 05 s
Speaker 3	Slides 6–7	Explainable priority, officer console	≈ 1 min 50 s
Speaker 4	Slides 8–10 + Q&A	Results, status, close	≈ 2 min

Handovers. Each speaker ends with one line that names the next speaker (marked → in the script). Replace [Speaker 2] and the others with your names.

Speak, don't read. Learn the first and last sentence of your part word for word; say the middle in your own words. The numbers are the part to get exactly right.

If you must cut to five minutes, drop the architecture fallback paragraph (slide 4) and the second half of slide 9.

Live demo. Hotspots, the "needs attention" strip and the suggested next step are not live until the backends are redeployed. Avoid clicking them, or redeploy first.

SLIDE 1 · TITLE

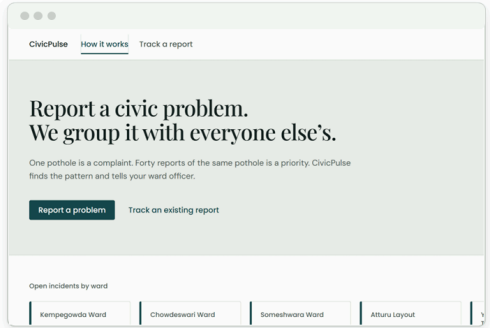
CPI PROJECT EVALUATION

CivicPulse

From scattered complaints to prioritised incidents

Citizens report problems in their own words. CivicPulse groups the reports that describe the same fault and tells the officer which to fix first, and why.

Suryansh · Abhishek · Devansh
Manya · Ria · Alaafiya



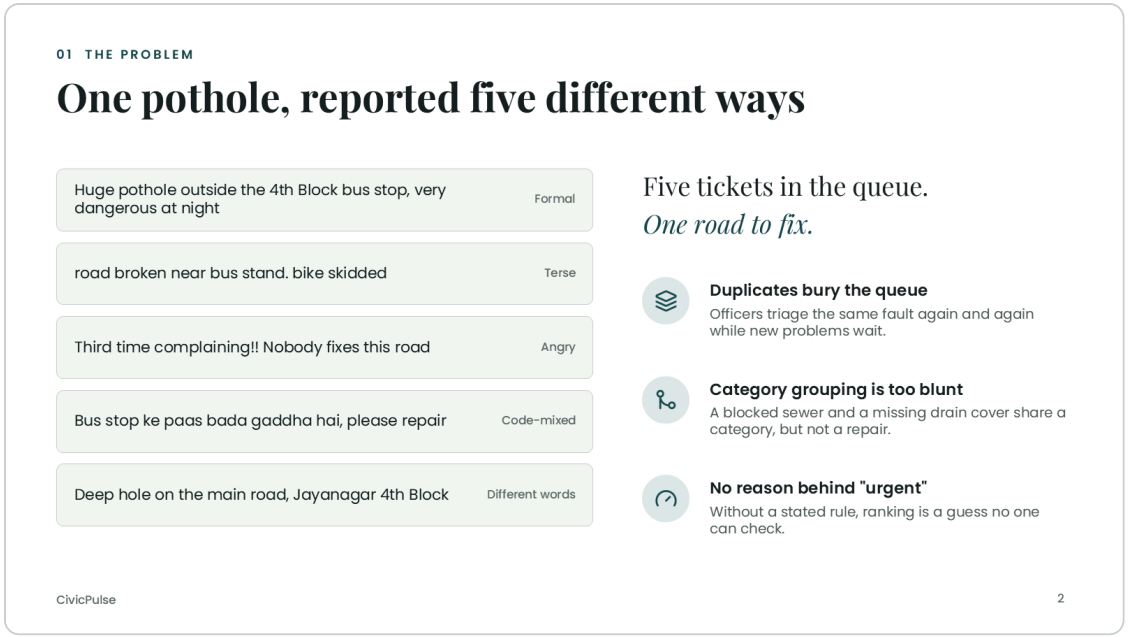
SPEAKER 1 · about 35 seconds

Good morning everyone. We are team CivicPulse. I am [name], and with me are [names].

CivicPulse is a civic-complaint platform built for Bengaluru. The idea fits in one line: citizens report problems in their own words, and we turn those reports into grouped, prioritised incidents that a ward officer can act on, with a written reason for every ranking.

What you see on the right is not a mock-up. It is the live site, running today.

SLIDE 2 · THE PROBLEM



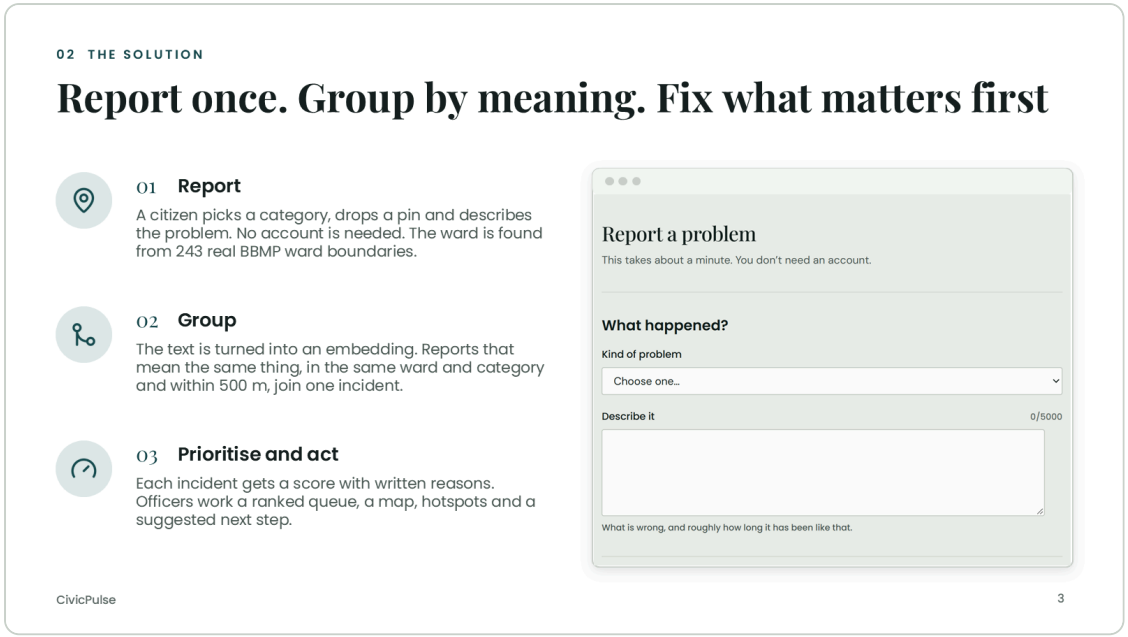
SPEAKER 1 · about 45 seconds

Let us start with the problem. Picture one pothole near a bus stop. Five people report it. One writes formally. One types five words. One is angry because it is their third complaint. One mixes Hindi and English. One describes it in completely different words.

To a city system, that is five tickets. To the officer, it is one road to fix.

That causes three problems. First, duplicates bury the queue: officers triage the same fault again and again while new problems wait. Second, grouping by category alone is too blunt: a blocked sewer and a missing drain cover share a category, but they need different repairs. Third, there is usually no stated reason why one complaint is “urgent” and another is not, so nobody can check the ranking.

SLIDE 3 · THE SOLUTION



SPEAKER 1 · about 50 seconds

So here is what CivicPulse does, in three steps.

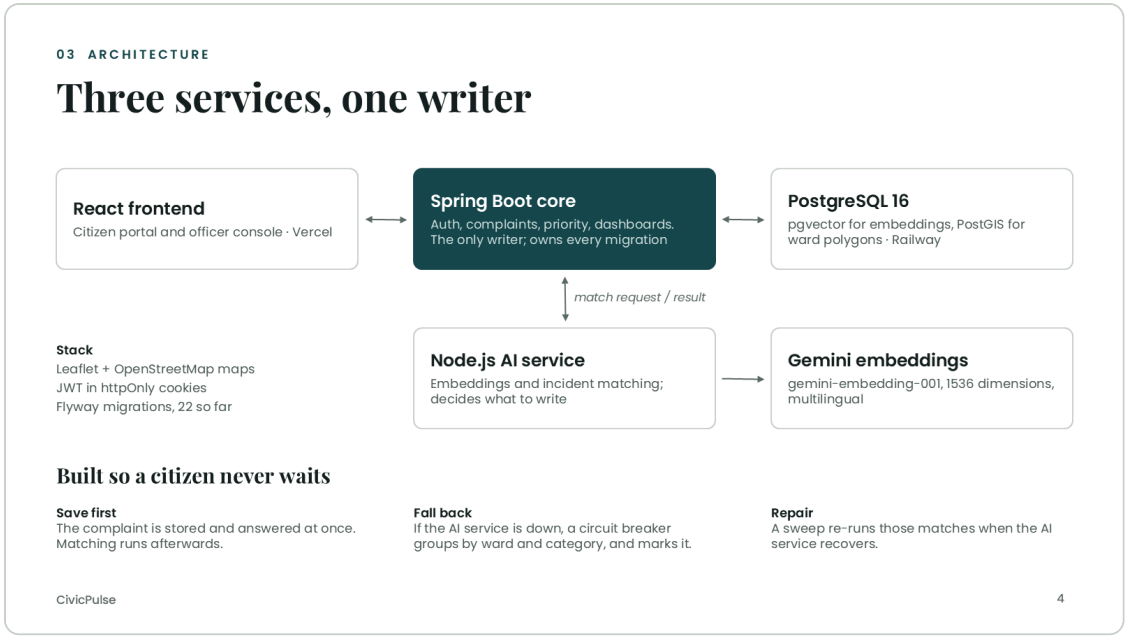
One: report. A citizen picks a category, drops a pin and describes the problem. No account is needed, and it takes about a minute. That is the real form on the right. We find the ward automatically from 243 real BBMP ward boundaries.

Two: group. We turn the text into an embedding, a list of numbers that captures meaning, so reports that say the same thing in different words land close together. If they are in the same ward and category, and within 500 metres, they join one incident.

Three: prioritise and act. Every incident gets a score with written reasons, and the officer works from a ranked queue on a map.

→ [Speaker 2] will now show you how it is built.

SLIDE 4 · ARCHITECTURE



SPEAKER 2 · about 60 seconds

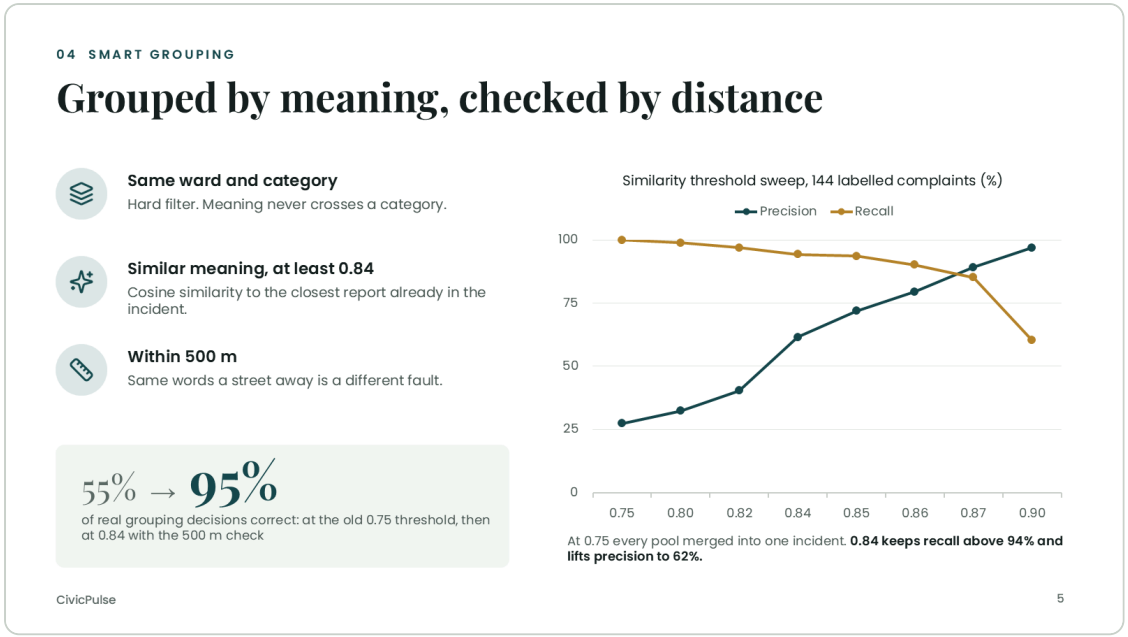
Thanks, [Speaker 1]. CivicPulse is three services.

The React frontend on Vercel serves both citizens and officers. At the centre is Spring Boot. It handles login, complaints, priority and the dashboards, and it is the only service allowed to write to the database. That is deliberate: with one writer, two services can never fight over the same record.

The Node.js service is our AI layer. It sends complaint text to Google's Gemini embedding model, finds the matching incident, and asks Spring to write the result. Everything lives in PostgreSQL, with pgvector for the embeddings and PostGIS for the ward maps.

One rule shaped all of this: a citizen should never wait for the AI. So the complaint is saved and confirmed first, and matching happens a second later in the background. If the AI service is down, a circuit breaker falls back to simple ward-and-category grouping and marks it, and a sweep redoes those matches once the service is back.

SLIDE 5 · SMART GROUPING



SPEAKER 2 · about 65 seconds

Now the heart of it: how we decide that two reports are the same problem. Three checks.

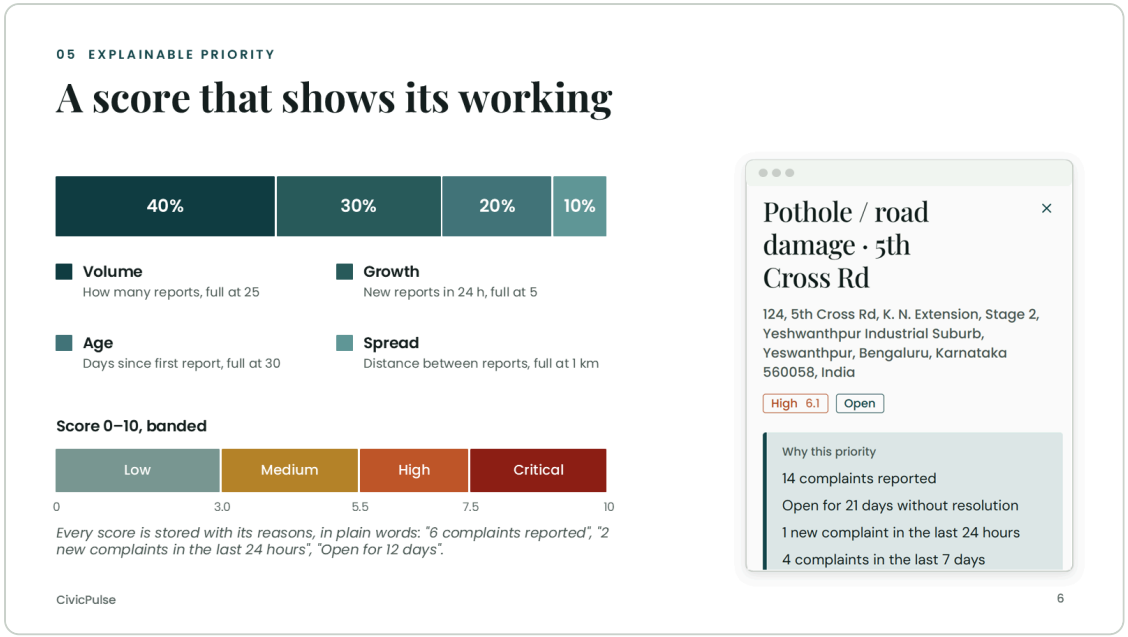
First, same ward and same category. That is a hard filter. Second, the meaning must be similar: a similarity score of at least 0.84 against the closest report already in the incident. Third, it must be within 500 metres, because the same words a few streets away usually mean a different fault.

We did not pick 0.84 by feel. We wrote 144 test complaints in which every ward and category had three different real problems, and we swept the threshold. That is the chart. At our original setting of 0.75, everything merged into one incident: recall was 100 percent, but precision only 27. At 0.84 we keep 94 percent recall, and precision more than doubles.

Then we checked real decisions from our live system. Before, 55 percent of grouping decisions were correct. With 0.84 and the distance check, it is 95 percent.

→ [Speaker 3] will take you through how we prioritise.

SLIDE 6 · EXPLAINABLE PRIORITY



SPEAKER 3 · about 60 seconds

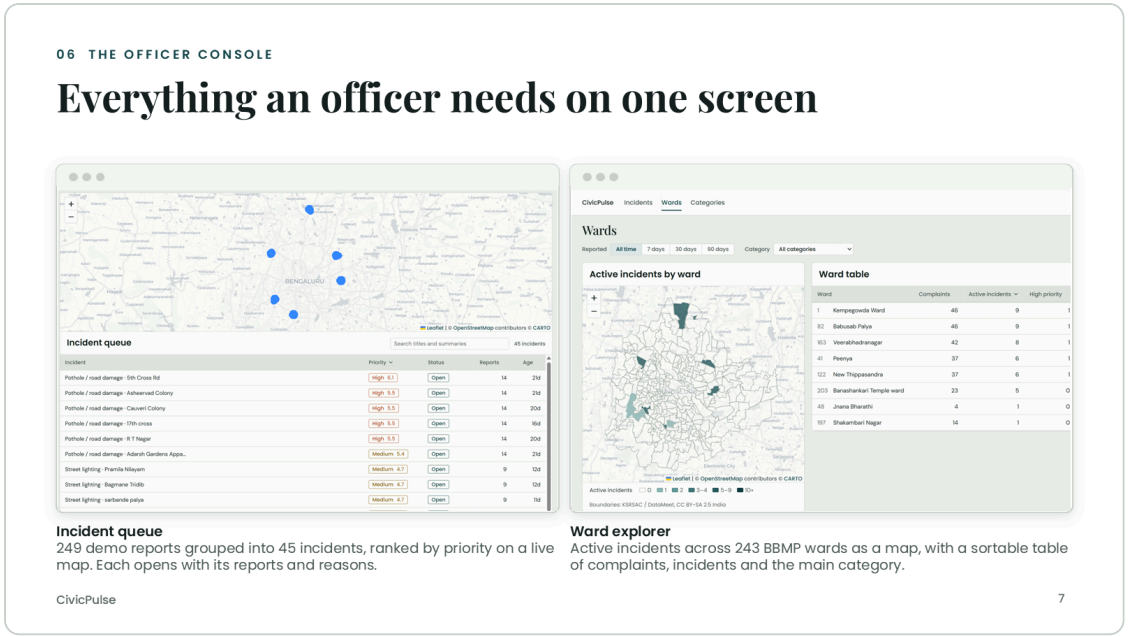
Thanks, [Speaker 2]. Once reports are grouped, the officer needs to know what to fix first.

We chose a rule, not a black-box model, for two reasons: there is no labelled data to train a model on, and an officer must be able to check the answer.

The score has four parts. Volume, 40 percent: how many people reported it. Growth, 30 percent: new reports in the last 24 hours, so a problem that is getting worse rises fast. Age, 20 percent: how long it has been open, up to 30 days. Spread, 10 percent: how far apart the reports are, up to a kilometre. That gives a score from 0 to 10, banded as low, medium, high and critical.

The important part is on the right. This is a real incident from the live system: a pothole on 5th Cross Road, scored High, 6.1. And it says why: 14 complaints, open for 21 days, one new complaint in the last day. Every score is stored with its reasons, in plain words.

SLIDE 7 · THE OFFICER CONSOLE



SPEAKER 3 · about 50 seconds

This is the officer console, live.

On the left is the incident queue. Our demo data has 249 reports, and CivicPulse has grouped them into 45 incidents, sorted by priority, each one placed on the map. Click a row and it opens with its member complaints and its reasons, and the officer can update its status.

On the right is the ward explorer: all 243 BBMP wards, shaded by how many active incidents each one has, next to a sortable table. An officer or a supervisor can see at a glance which wards are under pressure.

We have also built hotspot detection and a “needs attention” strip. They go live with our next backend deploy.

→ [Speaker 4] will now show you whether any of this actually works.

SLIDE 8 · RESULTS

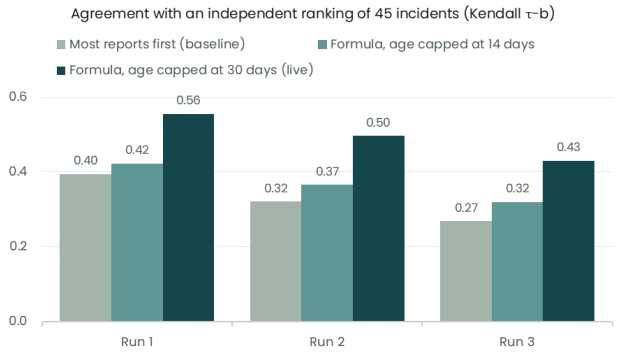
07 RESULTS

We measured it instead of assuming it

95%
of real grouping decisions correct (was 55%)

3 of 3
ranking runs where the formula beat the "most reports first" baseline

243
real BBMP ward boundaries, matched with PostGIS



Ranker: Gemini, briefed as a ward engineer and never shown the formula. A human officer ranking is the next check.

SPEAKER 4 · about 60 seconds

Thanks, [Speaker 3]. We wanted evidence, not assumptions, so we measured both halves of the system.

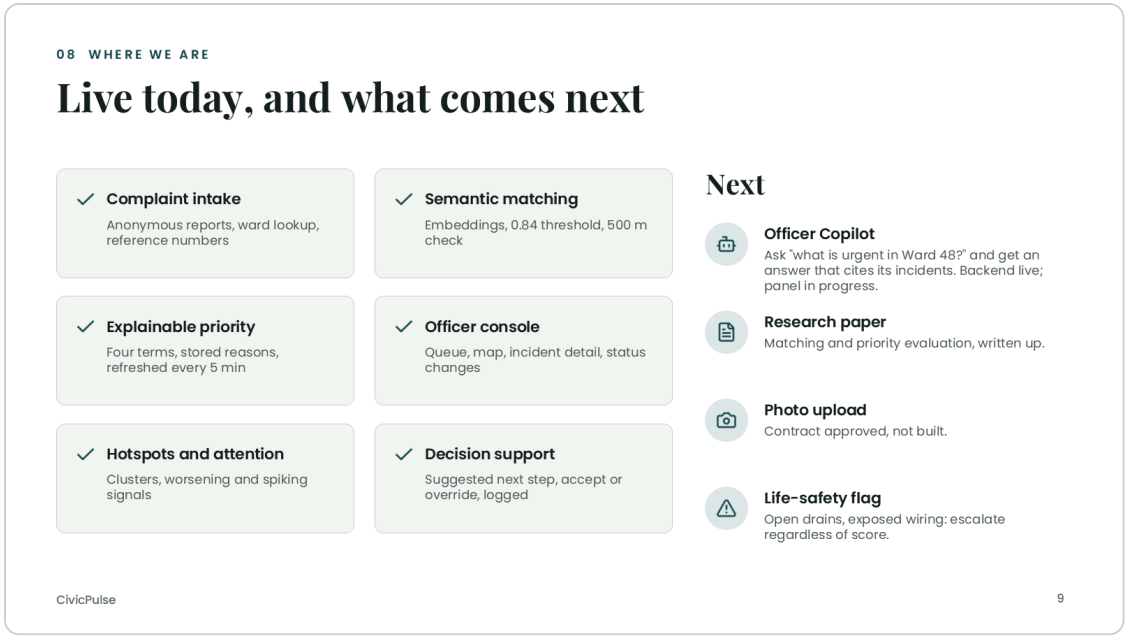
For grouping, you have already seen it: 95 percent of real grouping decisions are correct, up from 55.

For priority, no officer had ranked our queue yet, so we used a stand-in. We gave Gemini the same sheet an officer would see (reports, age, spread) but never our formula or our scores, and asked it to rank 45 incidents as a ward engineer would, three separate times. Then we measured how closely each method agreed with it.

The light bars are the naive approach: most reports first. The middle bars are our formula as first shipped, and it beat the naive baseline in all three runs. The dark bars show one change: letting age count up to 30 days instead of 14. That improved agreement in every run, so we applied it, and that is what is live now.

This ranker is a proxy. A real officer ranking is our next check.

SLIDE 9 · WHERE WE ARE



SPEAKER 4 · about 40 seconds

So where are we today?

Everything on the left is built and tested: complaint intake, semantic matching, explainable priority, the officer console, hotspots and attention signals, and decision support, which is a suggested next step the officer can accept or override, and every decision is logged.

Next is the Officer Copilot. An officer asks something like “what is urgent in Ward 48?” and gets an answer built only from our data, with every claim citing its source. The backend for it is already live. After that come our research paper, photo upload, and a life-safety flag, so things like open drains or exposed wiring escalate regardless of score.

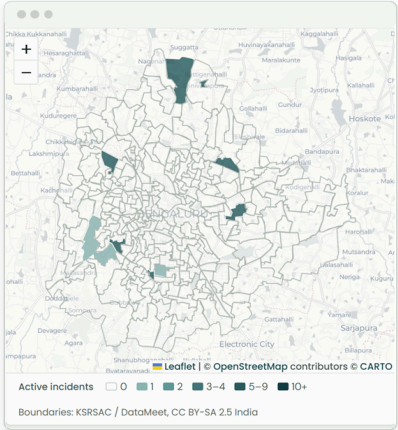
SLIDE 10 · THANK YOU

Thank you

Questions welcome

Try it live
civic-pulse-gamma-ivory.vercel.app

Source
github.com/Proj-CivicPulse/CivicPulse



SPEAKER 4 · about 20 seconds

To sum up: CivicPulse turns scattered complaints into grouped, prioritised incidents, and it can explain every decision it makes.

It is live at the address on screen, and you are welcome to try it.

Thank you. We are happy to take questions.

(Q&A: each speaker takes questions on their own slides.)

Likely questions

Question	Answer	Who
Why not machine learning for priority?	There is no labelled data to train on, and an officer has to be able to check the answer. A rule with stored reasons can be checked; a model cannot. Once officers rank real queues, we have the data to test anything smarter.	Speaker 3
Why 0.84 and not some other threshold?	It gave the best balance of precision and recall across both datasets: our 144 written test complaints and the real decisions from the live system. At 0.75 everything merged; above about 0.87 we start splitting real duplicates apart.	Speaker 2
What about two different problems at the same spot?	The distance check cannot separate them, but the meaning check does most of the work: a blocked sewer and a missing drain cover read differently. An officer merge/unlink tool for the rare mistake is on our backlog.	Speaker 2
Does it handle Kannada and Hindi?	Gemini's embedding model is multilingual, which is why we chose it. We measured one weak spot honestly: two code-mixed complaints about different problems can score as similar as two English ones about the same problem. Transliterating before embedding is the planned fix.	Speaker 2
Is this real complaint data?	The ward boundaries are real: 243 BBMP wards. The complaints are demo data we generated, because we have no access to a city feed yet. The system has a validated ingestion gate ready for a real feed.	Speaker 4
What does it cost to run?	Each complaint embedding costs a fraction of a rupee. Hosting is Vercel plus Railway.	Speaker 2
Why two backends instead of one?	Spring Boot holds the stable, transactional rules: login, complaints, priority. Node is where the AI work changes quickly. Keeping Spring as the only writer means the fast-moving part can never corrupt the data.	Speaker 2
What happens if Gemini is down?	The citizen never notices. The complaint is saved first; a circuit breaker groups it by ward and category instead, marks it, and a sweep redoes it properly when the service is back.	Speaker 2
Is citizen data sent to an AI?	Complaint text goes to Google's embedding model to compute similarity. The Copilot is designed so that resident complaint text is never sent to the chat model; it answers only from CivicPulse's own figures and records.	Speaker 4
How do you know the priority formula is right?	We tested it against an independent ranking of 45 incidents, three times. It beat "most reports first" every time, and the test told us to let age count up to 30 days, which we applied. The next check is a real officer ranking.	Speaker 4
Where are the hotspots?	Hotspot detection and the "needs attention" strip are built and tested. They go live with the next backend deploy.	Speaker 3